

MLPR Lab-15

This assignment is about using LSTM for Human Activity Recognition from smartphone data. Since we are dealing with sequential data, we would want to use LSTMs for this application.

Instructions:

- Submit the code with the outputs and the answers in one PDF.
- Approx. program execution time – 15 minutes

Dataset location: <https://archive.ics.uci.edu/ml/machine-learning-databases/00240/UCI%20HAR%20Dataset.zip>

The data was collected from 30 subjects aged between 19 and 48 years old performing one of six standard activities while wearing a waist-mounted smartphone that recorded the movement data. Video was recorded of each subject performing the activities and the movement data was labeled manually from these videos. The six activities performed were as follows:

1. Walking
2. Walking Upstairs
3. Walking Downstairs
4. Sitting
5. Standing
6. Laying

Step 1: Import libraries and associated functions

- `from numpy import mean`
- `from numpy import std`
- `from numpy import dstack`
- `from pandas import read_csv`
- `from keras.models import Sequential`
- `from keras.layers import Dense`
- `from keras.layers import Flatten`
- `from keras.layers import Dropout`
- `from keras.layers import LSTM`
- `from keras.utils import to_categorical`
- `from matplotlib import pyplot`

Step 2: Load each of the train and test datasets.

- There are three main signal types in the raw data: total acceleration, body acceleration, and body gyroscope. Each has 3 axes of data. This means that there are a total of nine variables for each time step.
- Further, each series of data has been partitioned into overlapping windows of 2.56 seconds of data, or 128 time steps (separated by whitespaces). This means that one row of data has $(128 * 9)$, or 1,152 elements.
- The signals are stored in the */Inertial Signals/* directory under the train and test subdirectories. Each axis of each signal is stored in a separate file, meaning that each of the train and test datasets have nine input files to load and one output file to load. We can batch the loading of these files into groups given the consistent directory structures and file naming conventions.
- Each of these files can be loaded as a 2D NumPy array:
`dataframe = read_csv(filepath, header=None, delim_whitespace=True)`
- We can then load all data for a given group (train or test) into a single three-dimensional NumPy array (by stacking up the 2D arrays along the third dimension - *features*), where the dimensions of the 3D array would be [*samples, time steps, features*]. To make this clearer, there are 128 time steps and nine features, where the number of samples is the number of rows in any given raw signal data file.

Step 4: Load target data.

- The output data is defined as an integer for the class number.
- First, we zero-offset the class number by subtracting 1 from all the output values.
- One hot encode these class integers so that the data is suitable for fitting a neural network multi-class classification model. We can do this by calling the [to_categorical\(\) Keras function](#).

Step 5: Fit and evaluate an LSTM model:

- We can define a function named *evaluate_model()* that takes the train and test dataset, fits a model on the training dataset, evaluates it on the test dataset, and returns an estimate of the model's performance.
- First, we must define the LSTM model using the Keras deep learning library. The model requires a three-dimensional input with [*samples, time steps, features*]. This is exactly how we have loaded the data, where one sample is one window of the time series data, each window has 128 time steps, and a time step has nine variables or features.
- The output for the model will be a six-element vector containing the probability of a given window belonging to each of the six activity types.
- We will define the model as having a single LSTM hidden layer. This is followed by a dropout layer intended to reduce overfitting of the model to the training data. Finally, a dense fully connected layer is used to interpret the features extracted by the LSTM hidden layer, before a final output layer is used to make predictions.
- The efficient Adam version of stochastic gradient descent will be used to optimize the network, and the categorical cross entropy loss function will be used given that we are learning a multi-class classification problem.

- The model is fit for a fixed number of epochs, in this case 15, and a batch size of 64 samples will be used, where 64 windows of data will be exposed to the model before the weights of the model are updated.
- Once the model is fit, it is evaluated on the test dataset and the accuracy of the fit model on the test dataset is returned (accuracy values should be about 89%).
- Evaluate the model 10 times and report the accuracy values and also the mean accuracy value.

REPORT QUESTIONS

1. Why was this problem suitable for LSTM and not just any other machine learning algorithm?
2. Is a many-to-many model a more real-world scenario for this problem or a many-to-one model and why?
3. Explain the meaning of “Short Memory” and “Long Memory” in context of LSTMs.
4. What are the three parts of an LSTM node and their functions?
5. What problems to RNNs face that LSTMs are able to adapt to fix?

DELIVERABLES: Items to be uploaded on LMS-

Main code file along with the complete code, all the outputs, and the report answers in one file in PDF format.